第六章 图

2022年10月2日 星期日 23:30

图的概念

基本概念

定义	表示
• 图=顶点+边	• 图G记为G=(V,E)
示例	**注意**
• G=(V,E) • V={1,2,3,4,5,6} • E={(1,2),(2,3),(3,4),(4,5),(1,5),(5,6),(2,6),(3,6)}	• 线性表有空表，树有空树，但图没有空图 • 图的顶点集V一定非空，但边集E可以为空 • |V|表图G中顶点的个数，也称图G的阶 • |E|表图G中边的个数

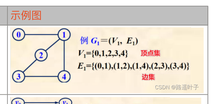
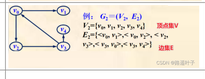
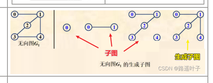
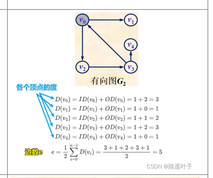
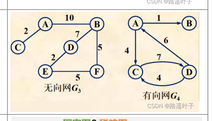
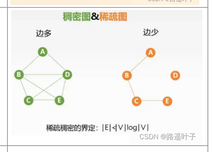


无向图的实例

常考结论

无向图边数*2 = 各顶点度数之和 有向图边数 = 各顶点的入度之和 = 各顶点的出度之和
一个连通图的生成树是一个极小连通子图，是无环的
完全无向图：边数为$\frac{n(n-1)}{2}$ 完全有向图：边数为 $n(n-1)$
对于一个有n个顶点的图 若是连通无向图，其边的个数至少为n-1（边最少即构成一棵树的情形） 若是强连通有向图，其边的个数至少是n（边最少即构成一个有向环的情形）

常见术语

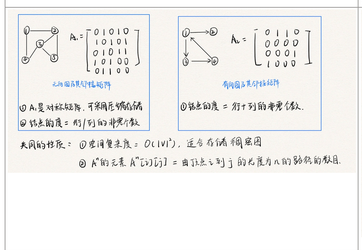
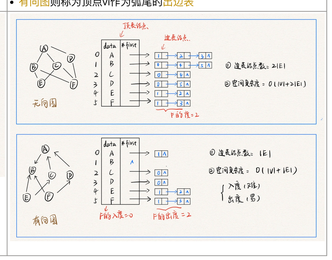
	定义	示例图
1、无向图	无向图：全部由无向边构成的图	
2、有向图	有向图：全部由有向边构成的图	
3、简单图、多重图	简单图：不存在重复边，不存在顶点到自身的边 多重图：两个顶点之间的边数大于1	
4、完全图(简单完全图)	完全无向图：边数为$\frac{n(n-1)}{2}$ 完全有向图：边数为 $n(n-1)$	
5、子图、生成子图	G的子图：所有的顶点和边都属于图G的图 G的生成子图：含有G的所有顶点的子图	
6、连通，连通图，连通分量 【无向图】	v和w连通：无向图中,v到w的路径存在 连通图：图中任意两个顶点都是连通的 连通分量：无向图中的极大连通子图	
7、强连通图，强连通分量 【有向图】	v和w强连通：有向图中,从v到w和从w到v都有路径 强连通图：图中任何一对顶点都是强联通的 强连通分量：有向图中的极大连通子图	
8、生成树，生成森林	生成树：包含图中全部顶点的一个极小连通图 生成森林：非连通图中,连通分量的生成树构成了非连通图的生成森林	
9、顶点的度，入度，出度	顶点的度Degree：图中与该顶点相关联边的数目 入度：指向该顶点的边的数目 出度：从该顶点出去的边的数目 顶点的度 = 出度 + 入度 对于具有n个顶点, e条边的有向图, 出度和=入度和=e 对于具有n个顶点, e条边的无向图, 度和=2e	
10、边的权和网	权Weight：边上的数值 网Network：边上标识权的图	
11、稠密图，稀疏图	稠密图：边多 稀疏图：边少	
12、路径，路径长度和回路	路径Path：在一个图中，路径是从顶点u到顶点v所经过的顶点序列 路径长度：该路径上边的数目 回路：第一个顶点和最后一个顶点相同的路径	
13、简单路径，简单回路	简单路径：顶点不重复出现的路径 简单回路：除第一个和最后一个顶点，其余顶点不重复出现的回路	
14、距离	从u到v的距离：从u到v的最短路径长度	
15、有向树	有向树：一个顶点的入度为0，其余顶点的入度均为1的有向图	

图的存储和基本操作

常考结论

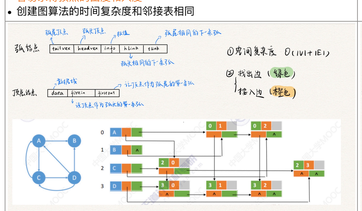
• 求有向图节点的入度，必须遍历整个邻接表
• 有向图邻接表中，删除某个顶点的所有边的时间复杂度 = O(n+e)
• 一个图的领接矩阵表示唯一，邻接表表示不唯一

图的存储方式

1、领接矩阵	2、邻接表
• 用两个数组来表示图 • 一个一位数组存储图中顶点信息 • 一个二维数组存储图中的边或弧的信息	• 领接表是由两部分组成 • 顶点用一个一维数组存储 • 每个顶点的所有领接点构成一个线性表 • 由于领接点的个数不定，所以用单链表存储 • 无向图称为顶点vi的边表 • 有向图则称为顶点vi作为弧尾的出边表
	

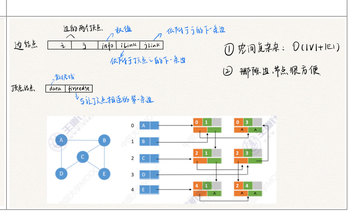
3、十字链表【针对有向图】

- 把邻接表和逆邻接表整合在了一起
- 既容易找到以vi为尾的弧，也容易找到以vi为头的弧
- 容易求得顶点的出度和入度
- 创建图算法的时间复杂度和邻接表相同



4、领接多重表【针对无向图】

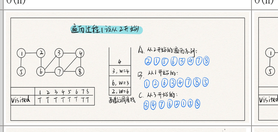
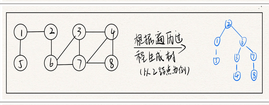
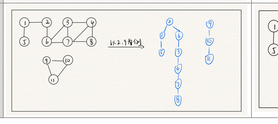
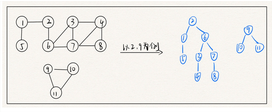
- 仿照十字链表的方式，对边表结点的结构进行一些改造
- 同一条边在邻接表中用两个结点表示，而在邻接多重表中只有一个结点



图的基本操作

Adjacent(G,x,y)	• 判断是否存在边
Neighbors(G,x)	• 列出图中与结点x邻接的边
InsertVertex(G,x)	• 在图中插入顶点x
DeleteVertex(G,x)	• 在图中删除顶点x
AddEdge(G,x,y)	• 若无向边或有向边不存在，则向图中添加该边
RemoveEdge(G,x,y)	• 若无向边或有向边存在，则向图中删除该边
NextNeighbor(G,x,y)	• 假设图中顶点y是顶点x的一个邻接点 • 返回除y外的顶点x的下一个邻接点的顶点号 • 若y是x的最后一个邻接点，则返回1
Get_Edge_Value(G,x,y)	• 获取图中边对应的权值
Set_Edge_Value(G,x,y,v)	• 设置图中边对应的权值

图的遍历

	深度优先搜索DFS	广度优先搜索BFS
定义	• 相当于树的先序遍历 • 根节点就是任意出发的节点 • 子节点就是所有邻近的未访问过的节点	• 相当于树的层序遍历
时间复杂度 【N是顶点】 【E是边】	邻接表存储 $O(N+E)$ 领接矩阵存储 $O(N^2)$	邻接表存储 $O(N+E)$ 领接矩阵存储 $O(N^2)$
空间复杂度	$O(n)$	$O(n)$
遍历过程		
生成树		
生成森林		

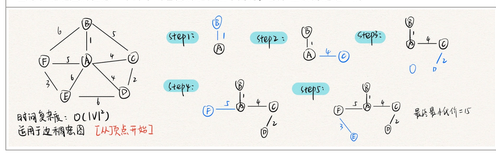
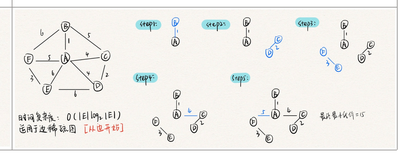
图的应用【重难点】

常考结论

• 最短路径一定是简单路径
• 求最短路径是允许图有环的
• 一个有向图的定点不能排成一个拓扑序列，表明其中存在一个顶点数目大于1的回路，该回路构成一个强连通分量
• 若有向无环图的拓扑序列唯一，不能唯一确定该图

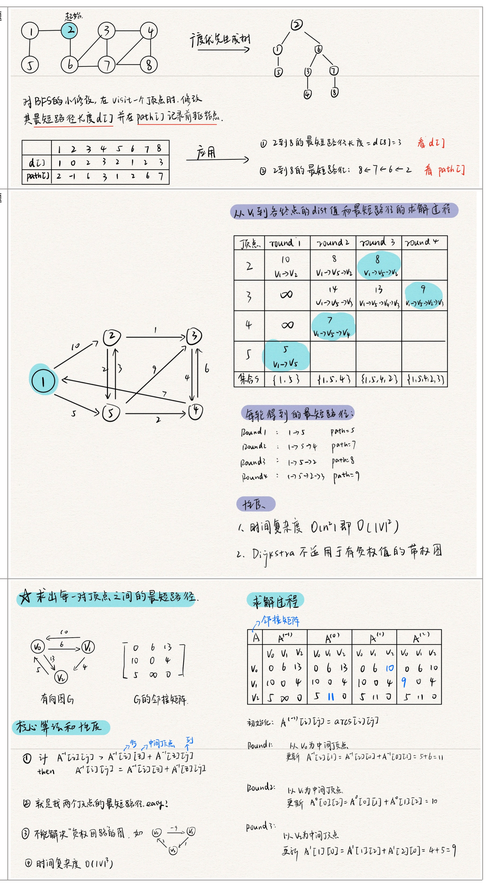
1、最小生成树

定义	• 图G的所有生成树中边的权值之和最小的树
性质	• 最小生成树并不是唯一的 • 当图G中的各边权值不相等时，最小生成树是唯一的 • 当G本身就是一棵树时，其最小生成树就是它本身 • 最小生成树的边的权值之和总是唯一的 • 最小生成树的边数 = 顶点数-1

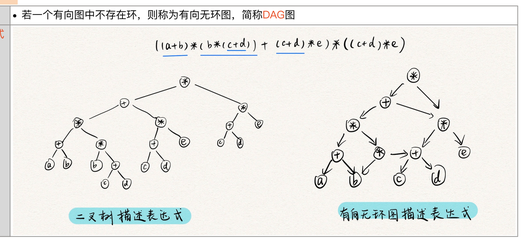
算法1：Prim算法	算法2：Kruskal算法
• 类似于寻找图的最短路径的Dijkstra算法【以自我为中心】 • 当带权连通图的任意一个环中所包含的边权值不同时，最小生成树是唯一的	• 按权值的递增次序选择合适的边来构造最小生成树【不以自我为中心】
	

2、最短路径

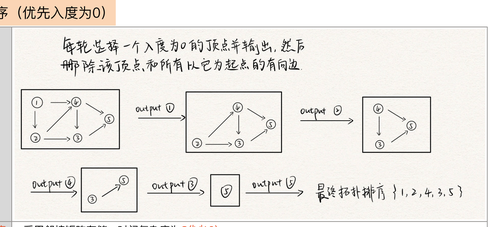
BFS	• 求无权图的单源最短路径问题
Dijkstra	• 求有向图的单源最短路径问题
Floyd算法	• 求任意两顶点间的最短路径 【不怎么考】



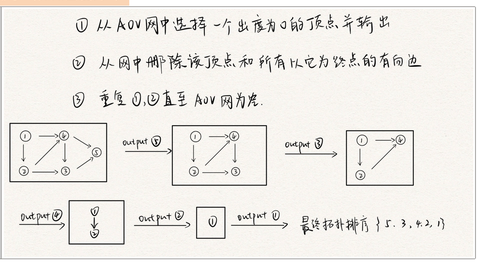
3、有向无环图描述表达式

有向无环图定义	• 若一个有向图中不存在环，则称为有向无环图，简称DAG图
利用有向无环图描述表达式	

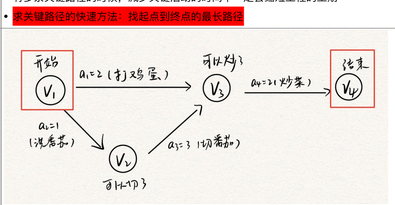
4、拓扑排序（优先入度为0）

核心算法	
时间复杂度	• 采用邻接矩阵存储，时间复杂度为O(|V|^2) • 采用邻接表存储，时间复杂度为O(|V|+|E|)

5、逆拓扑排序（优先出度为0）

核心算法	

6、关键路径

常考点	• 计算最早开始时间和最晚开始时间 • 关键路径是指权值之和最大而非边数最多的路径 • 无论存在一条或是多条关键路径，增加任一关键活动的时间都会延长工程的工期 • 只有一条关键路径的时候，减少关键活动的时间会缩短工程的工期 • 有多条关键路径的时候，减少关键活动的时间不一定会缩短工程的工期 • 求关键路径的快速方法：找起点到终点的最长路径
AOE网	
计算过程	